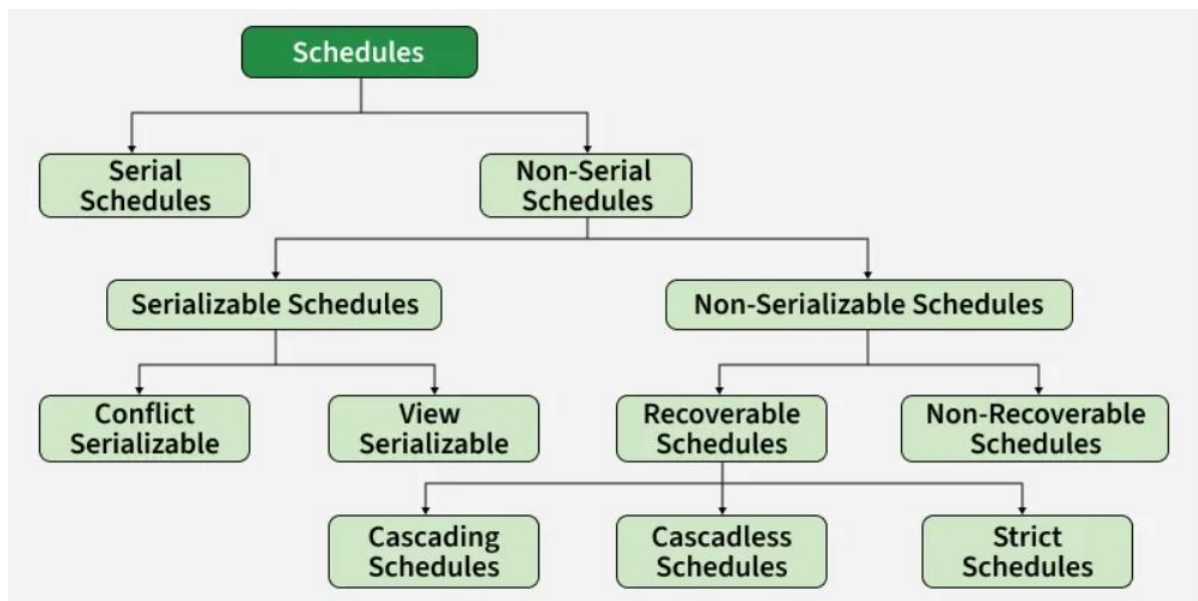


Scheduling is the process of determining the order in which transactions are executed. When multiple transactions run concurrently, scheduling ensures that operations are executed in a way that prevents conflicts or overlaps between them.



Types of Schedules in DBMS

1. Serial Schedule

Schedules in which transactions execute one after another without interleaving, i.e., no transaction starts until a running transaction has ended are called serial schedules.

- Only one transaction runs at a time.
- No concurrency issues.
- Simple and always consistent.

Example: Consider the following schedule involving two transactions T_1 and T_2 .

T ₁	T ₂
R(A)	
W(A)	
R(B)	
	W(B)
	R(A)
	R(B)

Serial Schedule

In the above table R(A) and W(A) denotes that a read and write operation is performed on some data item 'A' respectively, denoting a serial schedule since the transactions perform serially in the order **T₁ → T₂** .

2. Non-Serial Schedule

Transactions execute in an interleaved manner in Non-Serial Schedule.

- Allows concurrency.
- Must be checked for correctness.
- May or may not be serializable.

Non-serial schedules are of two types:

Serializable Scheduling (Concurrency Control)

A non-serial schedule that behaves like a serial schedule.

Why Serializable Schedules?

- Maintain consistency.
- Avoid anomalies.
- Improve performance by allowing concurrency.

There are two types of serializable schedules:

i. Conflict Serializable: A schedule is [conflict serializable](#) if it can be converted into a serial schedule by swapping non-conflicting operations.

Two operations conflict when:

- They belong to different transactions.
- They operate on the same data item.
- At least one operation is a write.

ii. View Serializable: A Schedule is called [view serializable](#) if:

- Transactions read the same initial values.
- Transactions read values written by the same transactions.
- The final write is by the same transaction as the serial schedule.

Note: Conflict-serializable schedules are a subset of view-serializable schedules. (Conflict-serializable $\subset$ View-serializable.)

Non-Serializable Scheduling

Schedules that do not preserve serial equivalence and may lead to inconsistencies if not handled carefully.

They are classified into the following types:

i. Recoverable Schedule: [Recoverable schedules](#) are those in which a transaction commits only after all transactions whose values it has read have also committed.

- Key Point: If T2 reads a value written by T1, then T2 must commit after T1.

Example: Consider the following schedule involving two transactions T₁ and T₂.

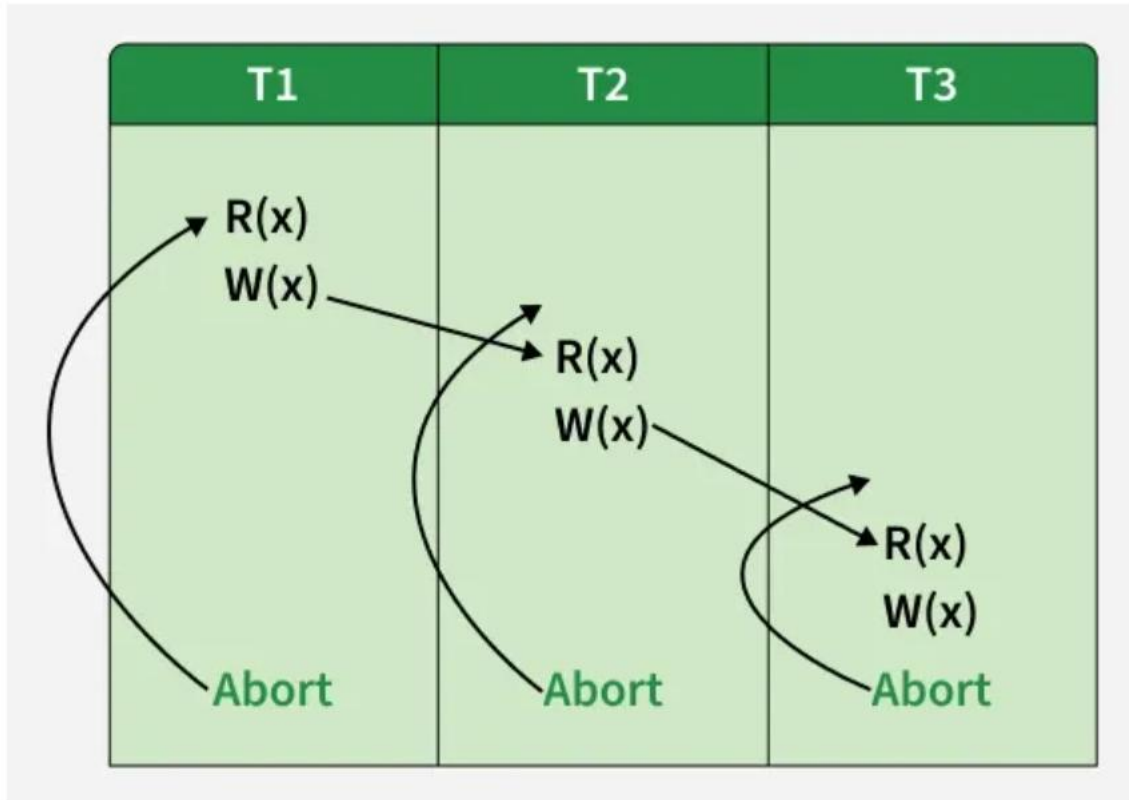
T ₁	T ₂
R(A)	
W(A)	
	W(A)
	R(A)
Commit	
	Commit

Recoverable Schedule

This is a recoverable schedule since T₁ commits before T₂, that makes the value read by T₂ correct. There can be three types of recoverable schedule:

ii. **Cascading Schedule:** A schedule where:

- Failure of one transaction forces all dependent transactions to abort.
- Happens when transactions read uncommitted data.



Cascading Abort

iii. **Cascadeless Schedule:** A schedule where:

- A transaction only reads committed data.
- Prevents cascading aborts.
- Key Point: T2 can read the value written by T1 only after T1 commits.

Example: Consider the following schedule involving two transactions T₁ and T₂.

T ₁	T ₂
R(A)	
W(A)	
	W(A)
Commit	
	R(A)
	Commit

Cascadeless Schedule

This schedule is cascadeless. Since the updated value of **A** is read by T₂ only after the updating transaction i.e. T₁ commits.

Example: Consider the following schedule involving two transactions T₁ and T₂.

T ₁	T ₂
R(A)	
W(A)	
	R(A)
	W(A)
abort	
	abort

It is a recoverable schedule but it does not avoid cascading aborts. It can be seen that if T₁ aborts, T₂ will have to be aborted too in order to maintain the correctness of the schedule as T₂ has already read the uncommitted value written by T₁.

iv. Strict Schedule: It's a stronger form of cascadeless schedule that follows the following rules:

- A transaction cannot read or write a data item written by another transaction until that transaction commits or aborts.
- Prevents dirty reads and dirty writes.

Example: Consider the following schedule involving two transactions T_1 and T_2 .

T_1	T_2
R(A)	
	R(A)
W(A)	
Commit	
	W(A)
	R(A)
	Commit

Strict Schedule

This is a strict schedule since T_2 reads and writes A which is written by T_1 only after the commit of T_1 .

v. Non-Recoverable Schedule: A schedule where:

- A transaction commits after reading **uncommitted data** from another transaction.
- If the writer transaction later aborts, the schedule becomes invalid.
- Key Point: These schedules must be avoided since they lead to inconsistencies.

Example: Consider the following schedule involving two transactions T_1 and T_2 .

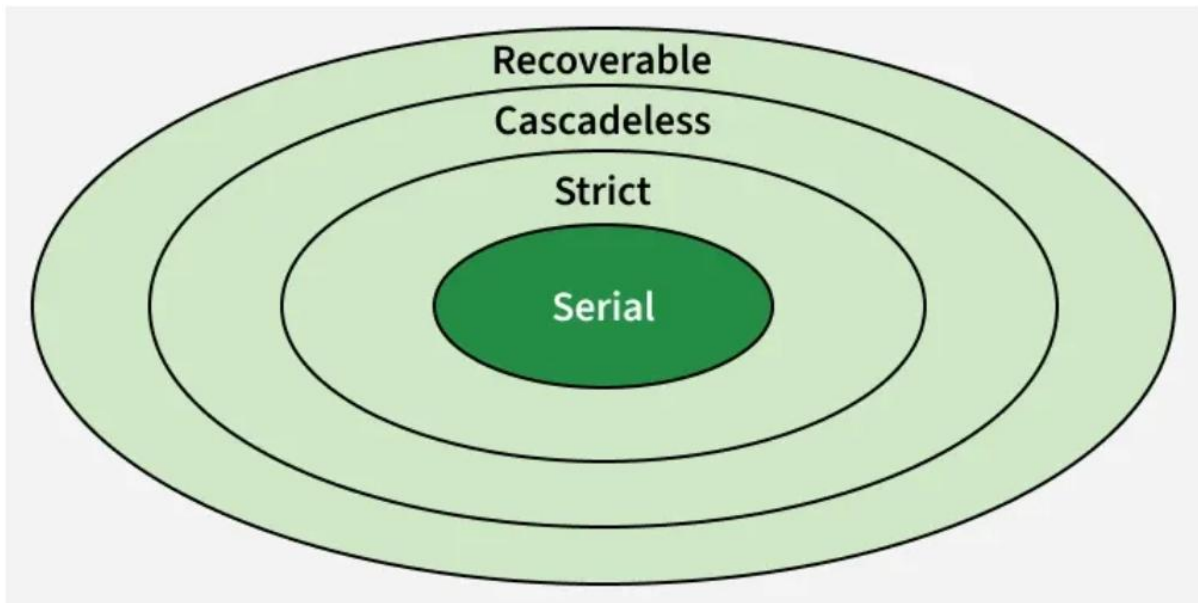
T_1	T_2
R(A)	
W(A)	
	W(A)
	R(A)
	Commit
abort	

Non-Recoverable Schedule

T_2 read the value of A written by T_1 , and committed. T_1 later aborted, therefore the value read by T_2 is wrong, but since T_2 committed, this schedule is non-recoverable.

Summary of Relationships

- Cascadeless schedules are stricter than recoverable schedules or are a subset of recoverable schedules.
- Strict schedules are stricter than cascadeless schedules or are a subset of cascadeless schedules.
- Serial schedules satisfy constraints of all recoverable, cascadeless and strict schedules and hence is a subset of strict schedules.



Relationship Venn Diagram of Types of Schedules